МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: Е. О. Максимов
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: Приобретение практических навыков в:

- Управление процессами в ОС
- Обеспечение обмена данных между процессами посредством каналов

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант: 14

Задание варианта: Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода. Child1 переводит строки в нижний регистр. Child2 убирает все задвоенные пробелы

Метод решения

Код данной лабораторной работы был написан так, чтобы его можно было запускать как на операционных системах POSIX, так и на Windows. Это было сделано благодаря реализации общего интерфейса для взаимодействия с операционной системой. При запуске основной программы помимо родительского процесса запускаются два дочерних, все процессы связаны между собой через 3 pipe'а, с системным вводом и выводом взаимодейтсвует только родительский процесс, ввод и вывод дочерних процессов перенаправляются в pipe'ы

Описание программы

Общий интерфейс взаимодействия с операционной системой описан в заголовочном файле src/os_interaction/basic/os.h. При компиляции программы в ней выбирается та или иная реализация требуемых в работе функций, файлы с реализацией этих функций под разные системы находятся в папках src/os_interaction/POSIX и src/os_interaction/Windows Основной код программы находится в файлах main.cpp, child1.cpp, child2.cpp

Осонвные структуры

```
1 struct FileHandleDummy { };
2 typedef FileHandleDummy* FileHandle;
3
4 struct ProcessHandleDummy { };
5 typedef ProcessHandleDummy* ProcessHandle;
```

Две структуры для описания HANDLE файлов и процессов для Windows.

```
struct Process {
    int pid = -1;
    ProcessHandle handle = nullptr;
};

struct File {
    int id = -1;
    FileHandle handle = nullptr;
};
```

Базовые структуры для описания процессов в Windows и POSIX.

```
struct ProcessStartInfo {

    std::string cmd;
    std::vector<std::string> args;

    File redirectStdin = {-1,nullptr};
    File redirectStdout = {-1,nullptr};

    ProcessStartInfo() = default;
};
```

Структура для описания параметров инициализации процесса (ключевые параметры - cmd, redirectStdin и redirectStdout).

Используемые системные вызовы

Windows:

- CreateProcess - создание процесса.
- TerminateProcess - принудительная остановка процесса.
- CreatePipe - создание pipe.
- GetStdHandle - получение стандартного потока ввода и вывода.
- Write - запись в поток.
- Read - чтение из потока.

POSIX:

- posix_spawn - создание процесса по сути является последовательным вызовом fork и execl (ещё можно между ними добавить dup2 или close), использован вместо fork и execl из-за похожести синтаксиса на CreateProcess, что повышает читаемость кода.
- kill - отправка процессу сигнала об окончании работы (SIGTERM или SIGKILL).
- pipe - создание pipe для POSIX.
- write - запись в поток.
- read - чтение из потока.

Результаты

В результате выполнения лабораторной работы была полученна программа, работающая с системными вызовами через общий интерфейс для ОС POSIX и Windows, расширяемая под другие системы, если потребуется.

В программе создаётся несколько процессов, общающихся через pipe, для совместной обработки входных данных.

Выводы

В результате выполнения работы было изучено создание процессов в 2 ОС и способ их общения через pipe'ы. Хотелось бы отметить три вещи, которые были узнаны в ходе выполнения работы:

- Функции для работы с процессами в POSIX куда более удобные и читаемые, чем в Windows (меньше параметров, не надо просматривать документацию долгое время)
- В стандартной библиотеке Windows существует значительный набор собственных типов (DWORD, HANDLE...)
- Система LaTeX не переносит символ _

Исходная программа

```
#pragma once

#include <algorithm>
#include <cassert>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <vector>
#include <string>


#if defined(_WIN32) || defined(WIN32) || defined(__CYGWIN__) || defined(__MINGW32__)
    || defined(__BORLANDC__)
    #define OS_WIN
    #define _USE_MATH_DEFINES
    #define NOMINMAX
    #include <Windows.h>
    #include <corecrt_math_defines.h>
    #include <windows.h>
    #include <processthreadsapi.h>
#elif defined(__unix__)
    #define OS_LINUX
    #include <stdint.h>
    #include <memory>
    #include <unistd.h>
    #include <errno.h>
    #include <signal.h>
    #include <spawn.h>
#endif

#define all(v) v.begin(), v.end()
```

Листинг 1: Используемые include, с выбором под конкретную ОС

```
#pragma once

#include "defines.h"

using BYTE = unsigned char;

namespace os {
struct FileHandleDummy { };
typedef FileHandleDummy* FileHandle;

struct ProcessHandleDummy { };
typedef ProcessHandleDummy* ProcessHandle;

struct Process {
    int pid = -1;
    ProcessHandle handle = nullptr;
};

struct File {
    int id = -1;
```

```
    FileHandle handle = nullptr;
};

struct ProcessStartInfo {

    std::string cmd;
    std::vector<std::string> args;

    File redirectStdin = {-1,nullptr};
    File redirectStdout = {-1,nullptr};

    ProcessStartInfo() = default;
};

Process create_proc(ProcessStartInfo& args);

void close_proc(Process proc);

void kill_proc(Process proc, uint32_t ret = -1);

bool create_pipe(File& readPipe, File& writePipe);

int pipe_write(File pipe, const void* buf, int count);

int pipe_read(File pipe, void* buf, int count);

int get_current_pid();

File get_stdin();

File get_stdout();

void close_file(File f);
} // namespace
```

Листинг 2: Основной интерфейс взаимодействия с ОС, используемый в коде основной программы

```
#pragma once

#include <iostream>

#include <ctime>

using BYTE = unsigned char;

#define Log(stream, level, category, message) \
    { \
        auto t = std::time(nullptr); \
        auto tm = *std::localtime(&t); \
        stream << std::put_time(&tm, "%H-%M-%S") << level << " " << category << " " <<
            message << "\n"; \
    }

#define LogMsg(category, message) \
    Log(std::cout, "MSG", category, message);
```

```
18
19 #define LogErr(category, message) \
20     Log(std::cerr, "ERR", category, message); \
21     Log(std::cout, "ERR", category, message);
22
23 #define LogWarn(category, message) \
24     Log(std::cout, "WARN", category, message);
```

Листинг 3: Макросы для логирования

```
1  #include "os.h"
2  #include "logs.h"
3
4
5  using namespace os;
6
7  #define SYS_CALL_AND_LOG(call) \
8      do { \
9          int RES = call; \
10         if (RES == -1) { \
11             std::stringstream error; \
12             error << #call << " failed. Error code: 0x" << std::hex << (int)errno; \
13             LogErr("os/posix", error.str()); \
14         } \
15     } while (0)
16
17 Process os::create_proc(ProcessStartInfo& info) {
18     posix_spawn_file_actions_t spfact{};
19     if ((info.redirectStdin.id != -1) || (info.redirectStdout.id != -1)) {
20
21         if (info.redirectStdin.id != -1) {
22             posix_spawn_file_actions_adddup2(&spfact, info.redirectStdin.id,
                   STDIN_FILENO);
23         }
24         if (info.redirectStdout.id != -1) {
25             posix_spawn_file_actions_adddup2(&spfact, info.redirectStdout.id,
                   STDOUT_FILENO);
26         }
27     }
28     char* argv[info.args.size()+2];
29     argv[0] = info.cmd.data();
30     for (std::size_t i = 1; i < info.args.size()+1; ++i) {
31         argv[i] = info.args[i-1].data();
32     }
33     argv[info.args.size()+1] = (char*) 0;
34     pid_t process_id{};
35
36     Process proc{0,nullptr};
37     int status = posix_spawn(&process_id, const_cast<char*>(info.cmd.data()), &spfact,
           NULL, argv, environ);
38     if (status != 0) {
39         std::stringstream error;
40         error << "Create process failed, error: 0x" << std::hex << status;
41         LogErr("os/posix", error.str());
42         proc.pid = -1;
43         return proc;
44     }
```

```
45
46      proc.pid = process_id;
47
48      return proc;
49  }
50
51  void os::close_proc(Process process) {
52      kill(process.pid, SIGTERM);
53  }
54
55  void os::kill_proc(Process process, uint32_t ret) {
56      kill(process.pid, SIGKILL);
57  }
58
59  bool os::create_pipe(File& outReadPipe, File& outWritePipe) {
60      int pipefd[2]{};
61      SYS_CALL_AND_LOG(pipe(pipefd));
62      outReadPipe.id = pipefd[0];
63      outWritePipe.id = pipefd[1];
64      return (outReadPipe.id + outWritePipe.id) > 0;
65  }
66
67  int os::pipe_write(File pipe, const void* buf, int count) {
68      const int fd = pipe.id;
69      int res = write(fd, buf, (size_t)count);
70      if (res == -1) {
71          std::stringstream error;
72          error << "Pipe write failed, error: 0x" << std::hex << errno;
73          LogErr("os/posix", error.str());
74      }
75      return res;
76  }
77
78  int os::pipe_read(File pipe, void* buf, int count) {
79      const int fd = pipe.id;
80      int res = read(fd, buf, (size_t)count);
81      if (res == -1) {
82          std::stringstream error;
83          error << "Pipe read failed, error: 0x" << std::hex << errno;
84          LogErr("os/posix", error.str());
85      }
86      return res;
87  }
88
89  int os::get_current_pid() {
90      return getpid();
91  }
92
93  File os::get_stdin() {
94      return File{STDIN_FILENO,nullptr};
95  }
96
97  File os::get_stdout() {
98      return File{STDOUT_FILENO,nullptr};
99  }
100
101 void os::close_file(File f) {
102     close(f.id);
```

```
}
```

Листинг 4: Реализация взаимодействия с ОС POSIX

```
#include "./os_api.h"
#include "os.h"
#include "logs.h"


using namespace os;

#define SYS_CALL_AND_LOG(call) \
    do { \
        BOOL RES = call; \
        if (RES == FALSE) { \
            std::stringstream error; \
            error << #call << " failed. Error code: 0x" << std::hex << GetLastError(); \
            LogErr("os/win_32", error.str()); \
        } \
    } while (0)

std::string generate_app_cmd(const char* cmd, const std::vector<std::string>& args) {
    static constexpr const char SEPARATOR = ' ';

    std::stringstream stream;
    stream << cmd;
    for (const std::string& arg : args) {
        bool needWrap = std::find(all(arg), SEPARATOR) != arg.end();

        if (needWrap) {
            stream << SEPARATOR << "\"" << arg << "\"";
        } else {
            stream << SEPARATOR << arg;
        }
    }
    return stream.str();
}

Process os::create_proc(ProcessStartInfo& info) {
    STARTUPINFO cif;
    ZeroMemory(&cif, sizeof(STARTUPINFO));

    BOOL redirectHandles = FALSE;
    if (info.redirectStdin.handle || info.redirectStdout.handle) {
        cif.dwFlags |= STARTF_USESTDHANDLES;
        redirectHandles = TRUE;

        if (info.redirectStdin.handle) {
            cif.hStdInput = info.redirectStdin.handle;
        }
        if (info.redirectStdout.handle) {
            cif.hStdOutput = info.redirectStdout.handle;
        }
    }

    BOOL isSuccess = FALSE;
```

```
53      PROCESS_INFORMATION pi;
54      ZeroMemory(&pi, sizeof(PROCESS_INFORMATION));
55
56      std::string cmd = generate_app_cmd(info.cmd.c_str(), info.args);
57
58      DWORD creationFlags = CREATE_DEFAULT_ERROR_MODE;
59      creationFlags |= CREATE_NO_WINDOW;
60
61      BOOL res = CreateProcess(nullptr,
62          const_cast<WCHAR*>(std::wstring(cmd.begin(),cmd.end()).data()),
63          nullptr,
64          nullptr,
65          redirectHandles,
66          creationFlags,
67          nullptr,
68          nullptr,
69          &cif,
70          &pi);
71
72      if (res == FALSE) {
73          std::stringstream error;
74          error << "Create process failed, error: 0x" << std::hex << GetLastError();
75          LogErr("os/win_32", error.str());
76      }
77
78      if (pi.hThread != nullptr && pi.hThread != INVALID_HANDLE_VALUE) {
79          CloseHandle(pi.hThread);
80      }
81      Process proc{0,nullptr};
82
83      if (pi.hProcess == INVALID_HANDLE_VALUE) {
84          std::stringstream error;
85          error << "Create process return invalid process handle, error: 0x" << std::hex
                << GetLastError();
86          LogErr("os/win_32", error.str());
87          return proc;
88      }
89
90      proc.handle = reinterpret_cast<ProcessHandle>(pi.hProcess);
91
92      return proc;
93  }
94
95  void os::close_proc(Process process) {
96      CloseHandle(process.handle);
97  }
98
99  void os::kill_proc(Process process, uint32_t ret) {
100     TerminateProcess(process.handle, static_cast<DWORD>(ret));
101 }
102
103 bool os::create_pipe(File& outReadPipe, File& outWritePipe) {
104     SECURITY_ATTRIBUTES sa;
105     ZeroMemory(&sa, sizeof(sa));
106     sa.nLength = sizeof(sa);
107     sa.bInheritHandle = TRUE;
108     sa.lpSecurityDescriptor = nullptr;
109
```

```
110     HANDLE pipeRead, pipeWrite;
111     SYS_CALL_AND_LOG(CreatePipe(&pipeRead, &pipeWrite, &sa, 0));
112     outReadPipe.handle = reinterpret_cast<FileHandle>(pipeRead);
113     outWritePipe.handle = reinterpret_cast<FileHandle>(pipeWrite);
114     return outReadPipe.handle && outWritePipe.handle;
115 }
116 
117 int os::pipe_write(File pipe, const void* buf, int count) {
118     DWORD countWritten;
119     BOOL RES{};
120     do {
121         BOOL RES = WriteFile(pipe.handle, buf, count, &countWritten, nullptr);
122         if (RES == FALSE) {
123             std::stringstream error;
124             error << "WriteFile failed. Error code: 0x" << std::hex << GetLastError();
125             LogErr("os/win_32", error.str());
126         }
127     } while (0);
128     if (RES) {
129         return countWritten;
130     }
131     return -1;
132 }
133 
134 int os::pipe_read(File pipe, void* buf, int count) {
135     DWORD countRead;
136     BOOL RES{};
137     do {
138         BOOL RES = ReadFile(pipe.handle, buf, count, &countRead, nullptr);
139         if (RES == FALSE) {
140             std::stringstream error;
141             error << "ReadFile failed. Error code: 0x" << std::hex << GetLastError();
142             LogErr("os/win_32", error.str());
143         }
144     } while (0);
145     if (RES) {
146         return countRead;
147     }
148     return -1;
149 }
150 
151 int os::get_current_pid() {
152     return (int)GetCurrentProcessId();
153 }
154 
155 File os::get_stdin() {
156     return File{0, (FileHandle)GetStdHandle(STD_INPUT_HANDLE)};
157 }
158 
159 File os::get_stdout() {
160     return File{0, (FileHandle)GetStdHandle(STD_OUTPUT_HANDLE)};
161 }
162 
163 void os::close_file(File f) {
164     CloseHandle(f.handle);
```

```
}
```

Листинг 5: Реализация взаимодействия с ОС Windows

```
#include <iostream>
#include <cstring>
#include "os_interaction/basic/os.h"

using namespace os;

int main() {
    File pipe1w, pipe1r;
    create_pipe(pipe1r, pipe1w);
    File pipe2w, pipe2r;
    create_pipe(pipe2r, pipe2w);
    File pipe3w, pipe3r;
    create_pipe(pipe3r, pipe3w);
    ProcessStartInfo proc_info1, proc_info2;
    proc_info1.cmd = "./lab1_exe_child1";
    proc_info2.cmd = "./lab1_exe_child2";
    proc_info1.redirectStdin = pipe1r;
    proc_info1.redirectStdout = pipe2w;
    proc_info2.redirectStdin = pipe2r;
    proc_info2.redirectStdout = pipe3w;
    Process child1 = create_proc(proc_info1);
    Process child2 = create_proc(proc_info2);
    std::string inp;
    bool running = true;
    while (running) {
        inp = "";
        char c{};
        while (c != '\n') {
            c = getchar();
            inp.push_back(c);
            if (inp == "exit\n") {
                running = false;
                break;
            }
        }
        if (!running) {
            inp[0] = 1;
            pipe_write(pipe1w, inp.data(), 1);
            break;
        }
        pipe_write(pipe1w, inp.data(), inp.size());
        char result{};
        while (result != '\n') {
            pipe_read(pipe3r, &result, 1);
            std::cout << result;
        }
    }
    kill_proc(child1);
    kill_proc(child2);
    close_file(pipe1r); close_file(pipe1w);
    close_file(pipe2r); close_file(pipe2w);
    close_file(pipe3r); close_file(pipe3w);
    return 0;
```

```
}
```

Листинг 6: Основная программа

```
#include <iostream>
#include <vector>
#include "os.h"

using namespace os;

int main() {
    std::vector<char> c{0};
    File input = get_stdin();
    File output = get_stdout();
    while (1) {
        os::pipe_read(input, c.data(), 1);
        if (c[0] == 1) {
            os::pipe_write(output, c.data(), 1);
            break;
        }
        if (('A' <= c[0]) && (c[0] <= 'Z')) {
            c[0] += 'a' - 'A';
        }
        os::pipe_write(output, c.data(), 1);
    }
    return 0;
}
```

Листинг 7: Программа первого дочернего процесса

```
#include <iostream>
#include <vector>
#include "os.h"

using namespace os;

int main() {
    std::vector<char> prev_c{0}, c{0};
    File input = get_stdin();
    File output = get_stdout();
    while (1) {
        prev_c[0] = c[0];
        os::pipe_read(input, c.data(), 1);
        if (c[0] == 1) {
            break;
        }
        if (!((c[0] == prev_c[0]) && (c[0] == ' '))) {
            os::pipe_write(output, c.data(), 1);
        }
    }
    return 0;
}
```

Листинг 8: Программа второго дочернего процесса